New Board Bring-up Procedure

Power-up • Smoke test • Validation — surviving Rev A

The discipline that turns delivered boards into working products

Audience: PCB engineers about to receive their first boards

Visual inspection • Power-up • Continuity • Functional • Documentation

Bring-up = Real Engineering

Reality check

Boards arrive — design
meets physics for first time.

Layered debug

Power → continuity →
clock → firmware → function.

Don't rush

Slow, methodical wins.
Fast = damaged boards.

Document

Every test, every result.
Future-you needs the data.

Rev A bugs

Industry assumes 1-3 revs.
Finding bugs early = on schedule.

Skill builder

Bring-up teaches more
than 100 textbook hours.

Pre-Bring-up Preparation

- Print 1:1 schematic — paper copy for marking, scribbling notes
- Have datasheets for top 5 ICs at hand (open in tabs)
- Variable-voltage bench supply (0-30V, current-limited)
- Multimeter — voltage + continuity modes
- Oscilloscope — minimum 100 MHz, 2-channel; better: 500 MHz, 4-channel
- Soldering iron + flux + tweezers for rework
- ESD-safe workspace + wrist strap
- Notebook + pen — every observation gets written down

Stage 1 — Visual Inspection (15 min)

- Under microscope or 10× loupe
- Check: all components placed correctly (orientation, polarity)
- Check: no solder bridges between fine-pitch pins
- Check: no missing components vs BOM
- Check: silkscreen ref-des matches schematic
- Check: connectors look clean, no bent pins
- Check: board edge clean (no copper exposure, no rough cuts)
- Photograph BEFORE powering up — record initial state

Stage 2 — Continuity + Smoke Test (30 min)

- BEFORE applying power — multimeter continuity tests
- VCC to GND: should be HIGH resistance (kΩ+) at every rail
- If LOW resistance (short): DO NOT POWER UP — find the short first
- Check ground continuity: all GND pins connected together
- Apply current-limited (~100 mA) low-voltage (~3.3V) first
- Watch supply current — sudden rise = smoke imminent → POWER OFF
- Use thermal camera — find hot ICs immediately
- Smell — burnt components have distinctive smell, act fast

Stage 3 — Power Rail Validation (30 min)

- Power up at nominal voltage (5V or whatever your input is)
- Measure EVERY power rail with multimeter: VCC_3V3, VCC_1V8, VCC_5V, etc.
- Voltage within ±5% of design target? OK to proceed
- Off by more: check feedback resistor divider values
- Scope ripple: should be < 50 mVpp on logic rails, < 10 mVpp on analog
- If ripple too high: check output cap value + ESR
- Measure quiescent current — compare to predicted
- Run for 5 minutes — recheck rails; should be stable

Stage 4 — Clock + Reset Validation (30 min)

- Scope the main MCU clock pin (XTAL_IN or similar)
- Should see sine wave near the crystal's frequency (e.g., 8 MHz, 32.768 kHz)
- If no oscillation: check crystal load caps + power on crystal IC
- Scope the MCU reset pin: should be HIGH for normal operation
- If stuck LOW: check reset chip + pull-up resistor
- Scope MCU power pins: should be at nominal supply voltage
- Verify any external clocks (e.g., for FPGA or PHY)

Stage 5 — Programming + Firmware (1-2 hr)

- Connect JTAG/SWD programmer (ST-Link, J-Link, DAPLink)
- Detect MCU: programmer should report device ID + flash size
- If no detection: check SWDIO/SWCLK + reset wiring + VCC to JTAG header
- Flash a simple LED-blinker firmware first — verify MCU runs
- LED toggles → core MCU + clock + power all OK
- No LED: debug printf via UART, check GPIO connection to LED
- Now run real firmware — observe each subsystem coming online
- Test in stages: comms → sensors → outputs → full functionality

Stage 6 — Functional Validation (variable)

- Test each major subsystem in isolation
- ADC channels: apply known voltage, read measured value
- Communication interfaces (UART, SPI, I^2C): loopback or use logic analyzer
- Sensor reads: place known stimulus (e.g., known temperature)
- Output drivers (LEDs, motors): toggle on/off, measure current
- Wireless: signal strength, range, packet rate
- Power consumption at typical operating mode
- Stress test: run for hours, watch for drift / heat issues

Stage 7 — Bug Documentation (continuous)

- Notebook entry per bug: date, observation, hypothesis, test result
- Bug categories: design error, manufacturing defect, firmware bug, intermittent
- Photograph anything visual
- Save scope screenshots (USB transfer is standard)
- Track in Git issues OR GitHub Project board
- Each bug: rev to fix in (Rev A workaround / Rev B fix)
- After bring-up: produce a 'Rev A bug report' document
- Use this as input for Rev B design

Common Bring-up Pitfalls

- Powering up before continuity test → magic smoke if short
- Not current-limiting the bench supply → blown components
- Forgetting to check VCC at the MCU itself (not just at the regulator output)
- Assuming firmware works because it ran on dev board → integrate one step at a time
- Not scoping at the IC pin — scoping further away misses local issues
- Replacing 'broken' parts without measuring — sometimes the issue is elsewhere
- Not documenting → repeating the same investigation next week
- Hurry = mistakes — bring-up is best done with calm focus

Bring-up Workflow Summary

- 1. Visual inspection (microscope)
- 2. Continuity + smoke test (multimeter + current limiter)
- 3. Power-up + rail validation (scope ripple, measure values)
- 4. Clock + reset validation
- 5. Programming + LED blink (verify MCU alive)
- 6. Subsystem testing (one at a time)
- 7. Integrated functional test
- 8. Documentation + bug list for Rev B

Key Takeaways

Key takeaways from this lecture

Pre-power continuity

Avoid smoke; find shorts first.

Current-limited supply

Sudden current = stop, debug.

Validate every rail

Multimeter + scope ripple.

LED blink milestone

MCU + clock + power = OK.

Subsystem at a time

Isolate the issue.

Document everything

Future you needs it.

Questions & Discussion

Ask anything — concepts, projects, sourcing, career.

References & Further Learning

- Hack a Day bring-up war stories
- EEVblog YouTube bring-up videos
- Robert Feranec — first board bring-up walk-throughs
- Phil's Lab — debug methodology videos
- Tektronix / Keysight scope training
- DigiKey / Mouser app notes on debug
- Local maker space — ask experienced engineers
- Open hardware projects' bring-up notes (Adafruit etc.)

Thank you • Build something this week.